

aslbench: A Benchmark for ASL Fingerspelling Recognition in Frontier Vision Language Models

2026-07-13

Table of contents

Executive summary	2
What the benchmark measures	2
Task and capability	2
Why this benchmark is novel	3
How the deliverable satisfies the requirements	3
Dataset and examples	3
Methodology	4
Dataset construction and sampling	4
Prompt templates	4
Model execution	5
Scoring and statistical design	5
Delivery process and implementation choices	6
Definitive results	6
Overall performance	6
Class-level pattern	7
Capability interpretation	8
Limitations and improvements	8
Ecological validity	8
Label identifiability	9
Sampling and uncertainty	9
Dataset scope and generalization	9
Prompt and model coverage	9
Benchmark integrity	9
Conclusion	9
Appendix: Definitive Run Detail	10
Run metadata	10
Full comparison	10
Confusion matrices	11
Paired significance tests	11
Per-model error detail	12
gpt-5.6-sol	12
gpt-5.6-terra	13

claude-opus-4.8	14
claude-sonnet-4.6	14

Executive summary

aslbench is a novel, interactive benchmark of fine-grained visual perception in frontier vision language models. It presents a model with a photograph from the ASL-HG dataset and asks it to identify one of 36 American Sign Language fingerspelling characters, digits 0 to 9 or letters A to Z. Predictions are parsed under a fixed response contract and scored by exact match against the dataset label. No human or model judge participates in scoring.

The definitive evaluation tested four frontier models on the same balanced set of 360 images. Accuracy ranged from **31.7% to 56.1%**, compared with a **2.8% random-choice baseline**. The best model, gpt-5.6-sol, correctly classified 202 images. Every model was meaningfully above chance and meaningfully below perfect, so the benchmark is neither trivial nor saturated. Exact paired McNemar tests found every pairwise accuracy difference significant after Holm correction at the 0.05 level.

The results show a substantial gap between broad visual competence and precise handshape recognition. Models handled distinctive static shapes such as V, 4, and 5 well, but all four failed every sampled J, M, N, and Z. Some errors reveal visual limitations, such as repeated confusion among compact fist-like shapes. Others expose limits in the benchmark itself, particularly motion-defined letters and character pairs whose still-image handshapes overlap.

What the benchmark measures

Task and capability

Each item contains one smartphone photograph of one or two hands forming a labeled character. The model must infer the character from subtle visual evidence:

- which fingers are extended, curled, crossed, or tucked;
- the thumb’s position relative to the fingers;
- palm orientation and viewpoint;
- occlusion among fingers;
- whether one or two hands are present.

This is a demanding case of fine-grained visual classification. The high-level object, a hand, remains almost constant across all 36 labels. The information that matters is concentrated in small, articulated structures that frequently overlap. Image resizing can remove useful detail, and a two-dimensional photograph can make depth relationships ambiguous. General-purpose VLM training data also contains far fewer carefully labeled hand configurations than common objects, scenes, and text. These conditions make fingers a persistent weak point for image generation and image understanding systems.

The capability matters beyond this dataset. Reliable hand and gesture perception is relevant to accessible interfaces, human-computer interaction, robotics, safety monitoring, and sign-language

technology. A model that recognizes a person and the surrounding scene but misses the exact hand configuration has not fully understood the image.

Why this benchmark is novel

aslbench is not a wrapper around an existing evaluation suite. It defines a new evaluation of general-purpose frontier VLMs on the January 2026 ASL-HG dataset from Pranto et al. Prior finger-spelling research has largely trained specialized computer-vision classifiers for sign recognition. To my knowledge after reviewing the available task and dataset literature, no published benchmark has compared frontier general-purpose VLMs on this exact 36-way ASL-HG task.

The novelty is the evaluation question and protocol: can an off-the-shelf frontier VLM, prompted through its normal multimodal interface and without task-specific training, resolve these hand-shapes? The source images are public, but the model comparison, prompt contract, leakage controls, paired sampling design, interactive runner, and statistical reporting are new.

How the deliverable satisfies the requirements

Novel. The benchmark targets a new model and dataset combination, with a purpose-built protocol for frontier VLMs. It does not call or repackage another benchmark. It tests fine-grained handshape perception directly.

Non-saturated. The definitive scores span 31.7% to 56.1%. All models beat the 2.8% chance baseline by a wide margin, while the best model still misses 158 of 360 items. The task therefore has room both to distinguish current models and to measure future progress.

Reproducible. The repository records the processed-dataset seed, definitive run seed, exact sampled item identifiers, prompt template, provider and model identifiers, package version, per-item responses, predictions, errors, and summaries. Every model in a run sees the identical images. The conda environment, provider configuration, tests, and Quarto export path are documented. A second evaluator can rebuild the dataset and obtain a comparable run, while the archived artifacts permit exact reanalysis of this run without making new API calls.

Quantitatively scored. Exact-match accuracy is the primary metric. Macro F1, bootstrap confidence intervals, normalized confusion matrices, class-level results, and exact paired significance tests provide complementary quantitative views. Parse failures and provider failures are tracked separately, so operational errors cannot disappear into the model score.

Interactive. The Dash application lets an evaluator choose subset size, prompt, and one or more provider-backed models; monitor progress; stop a run; compare results; inspect each image and raw response; manage run history; and export self-contained HTML or PDF reports.

Dataset and examples

ASL-HG contains 36 classes, 10 participants, and 1,000 source images per class. The photographs were captured by smartphone in natural indoor and outdoor settings. The repository’s processed subset contains 3,600 images, with 10 seeded images retained per participant and class. The definitive run sampled 10 images per class from that processed set.

The following examples were all included in the definitive run. They illustrate the task’s limited visual margins. The first image is the dataset’s two-handed digit 0; the others are the letters A, M, and O.



Figure 1: Digit 0. Two hands form a closed ring.



Figure 2: Letter A. A closed fist with the thumb alongside.



Figure 3: Letter M. A compact handshape with fingers folded over the thumb.

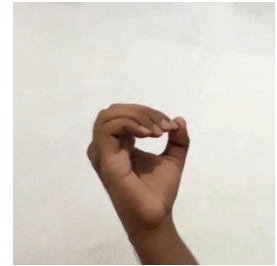


Figure 4: Letter O. One hand forms a rounded opening.

These examples also expose real model differences. Three models recognized the shown 0; `claude-sonnet-4.6` predicted 9. Three recognized the shown A; `claude-opus-4.8` predicted S. No model recognized the shown M. Three recognized the shown O; `claude-sonnet-4.6` predicted F.

Methodology

Dataset construction and sampling

The script `scripts/subset_dataset.py` creates the processed set from the public raw ASL-HG download. It samples 10 images for every participant within every class using seed 20260711, yielding 3,600 images with exact class and participant balance. The source folder is the authoritative label.

At run time, the evaluator deliberately chooses 1 to 10 images per class. There is no default selection. The runner samples without replacement using a newly generated seed and archives the selected item identifiers. Every selected model receives the same ordered item set, which makes cross-model comparisons paired rather than independent. The definitive run used seed 156084346 and the maximum UI setting of 10 images per class, for 360 total evaluations per model.

The filename encodes the answer, so leakage prevention is part of the benchmark design. OpenAI-compatible and Anthropic providers receive image bytes inline. The Copilot provider copies each image to a temporary neutral filename before attachment, disables tools, runs a single-item session, then destroys the session and temporary file. The model sees pixels, not the source path.

Prompt templates

The project includes three prompts to make prompt sensitivity explicit rather than silently treating one wording as the task itself.

1. **v1_zeroshot** states the task and response format with minimal guidance. It measures performance under the least scaffolding, but leaves more room for uncertainty about the label space and dataset-specific conventions.
2. **v2_class_list** lists all 36 valid outputs, directs attention to finger, thumb, palm, and hand-count evidence, and explains the dataset’s one-handed 0 versus two-handed 0 distinction. It standardizes task knowledge without prescribing a long reasoning process.
3. **v3_reasoning** adds a five-step visual analysis procedure. It can test whether explicit decomposition helps, but it also increases output length and introduces a reasoning-compliance variable alongside perception.

The definitive benchmark used **v2_class_list**. It is the best middle condition for a model comparison: more controlled than the minimal prompt, but less interventionist than mandatory step-by-step reasoning. The task is perception, so the final protocol supplies the label ontology and known dataset convention while leaving the model’s internal decision process unconstrained. All templates require the final line `ANSWER: <single character>`.

Model execution

The provider abstraction supports GitHub Copilot, Anthropic, OpenAI-compatible cloud APIs, and local OpenAI-compatible servers such as LM Studio or oMLX. Each model executes in its own background thread, while items remain sequential within a model. Responses, optional thinking text, latency, token counts, parse status, and provider errors are written incrementally. Atomic state files support progress polling and post-interruption recovery.

The definitive run compared these GitHub Copilot models:

- gpt-5.6-sol
- gpt-5.6-terra
- claude-opus-4.8
- claude-sonnet-4.6

All 1,440 calls completed. Every response parsed successfully, and no provider errors occurred.

Scoring and statistical design

The parser prefers the last valid `ANSWER: X` line and permits a bare single-character fallback. Letters are normalized to uppercase. An invalid or missing answer is incorrect and is also counted as a parse failure.

The primary metric is accuracy:

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{evaluated images}}.$$

Macro F1 gives each class equal weight, then combines class precision and recall. This matters even in a balanced run because a model may overpredict a few familiar labels. Accuracy and macro F1 receive deterministic 95% percentile bootstrap intervals from 1,000 item-level resamples.

Confusion matrices are normalized within each true class, making the diagonal equal to per-class recall. Pairwise model differences use exact two-sided McNemar tests on the shared items. Holm correction controls family-wise error across all six model pairs. This paired test is more appropriate than comparing independent confidence intervals because every model saw exactly the same images.

Delivery process and implementation choices

I used an iterative, model-assisted engineering workflow, with each model serving a different role and with human audit between stages.

1. I developed the benchmark concept through a back-and-forth conversation with Sonnet 5, then captured the agreed problem, constraints, and assessment requirements in `BRIEF.md`.
2. I gave that brief to Fable 5 to produce the detailed architecture in `PLAN.md`. I audited and refined the plan before implementation, especially around provider abstraction, filename leakage, long-running Dash work, reproducible sampling, and Quarto export.
3. I used Opus 4.8 to implement the first working version across the dataset, providers, runner, scoring, application, tests, and report layers.
4. After reviewing the statistical questions the benchmark should answer, I refined the metrics and figures. The finished product intentionally differs from the initial plan where a simpler or more defensible analysis emerged.
5. I used Sonnet 4.6 to refine the UI, make targeted behavior changes, and audit the codebase. I continued validating the implementation against the original requirements rather than treating generated code as final by default.

Important deviations from the plan include a chance baseline, macro F1 confidence intervals, exact paired McNemar tests with Holm correction, and per-class difference plots sorted by model advantage. These additions make effect size, uncertainty, and pairwise differentiation easier to interpret. The app also gained run cancellation, deletion with confirmation, stored thinking-text support, and interrupted-run recovery. Conversely, the final dashboard and report deemphasized some planned summary tables, such as participant bars and broad lists of hardest items, in favor of comparative figures and an item-level explorer. The underlying per-item and participant data remain archived for reanalysis.

The architecture retained the plan's most important boundaries. Benchmark logic lives in importable Python modules rather than callbacks or shell scripts. The app calls the same dataset, runner, scoring, and figure functions used by tests and reports. Run folders are durable intermediates, and Quarto renders those artifacts without rerunning models.

Definitive results

Overall performance

Model	Correct	Accuracy	95% accuracy CI	Macro F1	95% macro F1 CI
gpt-5.6-sol	202 / 360	0.561	0.511 to 0.611	0.517	0.468 to 0.549
gpt-5.6-terra	163 / 360	0.453	0.400 to 0.506	0.386	0.343 to 0.413
claude-opus-4.8	143 / 360	0.397	0.350 to 0.444	0.355	0.309 to 0.385
claude-sonnet-4.6	114 / 360	0.317	0.269 to 0.367	0.249	0.213 to 0.271
Uniform random choice	about 10 / 360	0.028	not applicable	0.028	not applicable

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

The ranking is consistent across accuracy and macro F1. Macro F1 is lower than accuracy for every model, especially claude-sonnet-4.6. This indicates that aggregate correctness is supported by strong performance on a subset of labels rather than uniform competence across the alphabet and digits.

Every paired difference is significant after Holm correction. The closest comparison is gpt-5.6-terra versus claude-opus-4.8; the adjusted value is 0.0446, with 55 shared images won only by Terra and 35 won only by Opus. The strongest model, gpt-5.6-sol, beats Terra on 67 discordant items while losing 28, and the adjusted value is 0.000234.

Class-level pattern

Across models, the easiest labels were V at 97.5% mean accuracy, 4 and 5 at 90.0%, and Y at 85.0%. The hardest were J, M, N, and Z, all at 0%. The labels 2 and 9 averaged 2.5%, while K also averaged 2.5%.

The errors are highly structured:

- 2 was read as V 9 or 10 times by every model. This is evidence of visual or label ambiguity, not random noise.
- J was read as I on all 10 images by both GPT variants; the still image omits the motion that distinguishes J.
- M and N were usually collapsed into S, reflecting difficulty resolving hidden thumb placement and finger overlap in compact handshapes.
- K was read as V 7 to 10 times, which suggests difficulty detecting the thumb's contact and depth relationship.

- E was frequently read as S, another compact-shape confusion.
- P was frequently read as G, consistent with an orientation-dependent distinction.

Unable to display output for mime type(s): text/html

Capability interpretation

The benchmark measures more than generic image understanding. All models are far above chance, showing that they possess meaningful ASL handshape knowledge and can apply it to unfamiliar photographs. Yet no model exceeds 57%, showing that this knowledge does not translate into robust fine-detail perception.

Model scale or family alone does not determine the error pattern. The four models differ substantially in aggregate score, but they share several dominant confusions. Shared failures on M, N, J, and Z point to common representational or task-level constraints. Model-specific differences, such as Sonnet’s 90% on 1 while Terra scores 0%, show that the benchmark also detects genuinely different class profiles.

The best model is better, not merely more compliant. All models have 0% parse-failure and provider-error rates. The score differences therefore come from classifications rather than formatting or infrastructure. Paired tests confirm that the ordering is not explained by each model happening to receive easier images.

Static spatial cues and motion cues separate cleanly. Distinctive static configurations such as V, 4, and 5 are strong. Motion-defined J and Z are universally wrong. That contrast is a useful diagnostic, but it also marks a boundary on what a single-frame benchmark can validly claim.

There is a shared hard core. All four models correctly classified 58 images, and all four missed 112. At least one model solved 248 of 360 images. This means 190 items produced disagreement among models, enough variation to support model comparison, while the 112 universally missed items identify a rich target for error analysis and future benchmark revision.

Limitations and improvements

Ecological validity

Fingerspelling still images are not an ecologically complete representation of ASL. ASL users communicate through continuous video with movement, facial expression, body position, spatial grammar, and transitions between signs. Fingerspelling is one component of the language, not ASL as a whole. A video benchmark would better reflect actual communication and would correctly represent movement-dependent letters such as J and Z.

I did not use video because it would be disproportionately expensive for this assessment. Video evaluation requires frame extraction or native video input, many more visual tokens, longer requests, and repeated inference across enough examples to estimate performance. It also introduces temporal sampling and alignment decisions that would substantially expand the benchmark’s

scope. With more time and budget, I would add a smaller, carefully controlled video track rather than replacing the reproducible still-image track.

Label identifiability

Some nominally different characters share or nearly share a static handshape. The near-universal 2 to V confusion is the clearest example. J and Z are defined partly by motion. Treating these cases as ordinary 36-way image errors can understate actual visual competence. A revised benchmark should annotate whether each class is statically identifiable, report a static-only core score, and reserve movement-defined labels for video.

Sampling and uncertainty

The definitive run uses 10 images per class, which is enough to reveal large effects but yields coarse 10-point steps in class accuracy. Runtime sampling is balanced by class but not explicitly stratified by participant, even though the processed pool is participant-balanced. The current item-level bootstrap also treats images as independent. A larger study should sample within class and participant, then use a hierarchical or clustered bootstrap to reflect repeated images from the same signers.

Dataset scope and generalization

ASL-HG contains 10 volunteers from one collection context in Dhaka, Bangladesh. Its natural backgrounds are useful, but one dataset cannot establish performance across cameras, signing styles, left-handed signers, age groups, disabilities, regions, or native signing proficiency. Participant-level accuracy variation in this run also suggests sensitivity to signer or capture conditions. Replication on independently collected datasets and a signer-held-out analysis would strengthen external validity.

Prompt and model coverage

The definitive run fixes one prompt, one provider path, and four models. That controls comparison, but it does not measure prompt sensitivity or separate model behavior from provider-side preprocessing. A fuller study would run all three prompts as a preregistered factorial comparison, include direct API and local open-weight models, repeat samples across seeds, and record cost and latency alongside accuracy.

Benchmark integrity

The dataset is recent, which lowers but does not eliminate contamination risk. A future version should maintain a private or newly collected holdout, publish hashes and manifests, add automated image-quality checks, and verify that neutralized attachments remain filename-safe across provider updates.

Conclusion

aslbench turns a recognizable VLM weakness into a reproducible, quantitative, and interactive evaluation. Its definitive run clearly differentiates four frontier models, with scores well above chance and well below saturation. The strongest evidence is not just the ranking; it is the struc-

tured pattern of success and failure across handshapes, the absence of parse or provider confounds, and the significant paired differences on identical images.

The benchmark also demonstrates the value of error analysis in benchmark design. Universal failures on movement-defined or visually overlapping labels are findings about both the models and the measurement instrument. The current version is useful as a static fine-grained perception benchmark. A next version should preserve that controlled core while separating static identifiability from temporal sign recognition.

Appendix: Definitive Run Detail

This appendix is the detailed run report for `aslbench-20260713-203909`. It is generated from the archived configuration and per-item result files using the same scoring and figure functions as the interactive application. It is included here in place of the standalone run report title so that the appendix remains part of a single coherent final report.

Run metadata

- **Run:** `aslbench-20260713-203909`
- **Started:** 2026-07-13T20:39:09
- **Images per class:** 10
- **Classes:** 36
- **Total images:** 360
- **Sample seed:** 156084346
- **Prompt template:** `v2_class_list`
- **Note:** GPT-5.6 Sol, GPT-5.6 Terra, Opus 4.8, Sonnet 4.6

Provider	Model	Slug
GitHub Copilot	<code>gpt-5.6-sol</code>	<code>copilot-gpt-5-6-sol</code>
GitHub Copilot	<code>gpt-5.6-terra</code>	<code>copilot-gpt-5-6-terra</code>
GitHub Copilot	<code>claude-opus-4.8</code>	<code>copilot-claude-opus-4-8</code>
GitHub Copilot	<code>claude-sonnet-4.6</code>	<code>copilot-claude-sonnet-4-6</code>

The task is 36-way single-character classification. Every model saw the identical sampled images, so model comparisons are paired. Filenames, which encode the true class, were never shown to the models.

Full comparison

metric	gpt-5.6-sol	gpt-5.6-terra	claude- opus-4.8	claude-son- net-4.6	Chance (1/36)
accuracy	0.561	0.453	0.397	0.317	0.028
macro_f1	0.517	0.386	0.355	0.249	0.028
parse_fail- ure_rate	0.000	0.000	0.000	0.000	0.000
provider_er- ror_rate	0.000	0.000	0.000	0.000	0.000

- **Accuracy:** the share of images labeled correctly.
- **Macro F1:** the class-balanced harmonic mean of precision and recall.
- **Parse failure rate:** the share of replies that could not be read as one valid character.
- **Provider error rate:** the share of calls that returned no usable model response.
- **Chance:** the expected score from uniform random choice among 36 labels.

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Confusion matrices

Unable to display output for mime type(s): text/html

Each row is a true character and each column is the prediction. Values are normalized within true character, so diagonal cells show recall and bright off-diagonal cells show systematic confusions.

Paired significance tests

Model A	Model B	Better	A only	B only	Discor- dant	p	p (Holm)	Signifi- cant
gpt-5.6- sol	claude- son- net-4.6	gpt-5.6- sol	105	17	122	0.000	0.000	yes
gpt-5.6- sol	claude- opus-4.8	gpt-5.6- sol	87	28	115	0.000	0.000	yes
gpt-5.6- terra	claude- son- net-4.6	gpt-5.6- terra	78	29	107	0.000	0.000	yes
gpt-5.6- sol	gpt-5.6- terra	gpt-5.6- sol	67	28	95	0.000	0.000	yes
claude- opus-4.8	claude- son- net-4.6	claude- opus-4.8	61	32	93	0.003	0.007	yes
gpt-5.6- terra	claude- opus-4.8	gpt-5.6- terra	55	35	90	0.045	0.045	yes

The exact two-sided McNemar test uses only discordant images, where one model is correct and the other is wrong. p (Holm) adjusts across every pairwise comparison. A result is significant when the adjusted value is below 0.05.

Per-model error detail

gpt-5.6-sol

Most-confused pairs

True	Predicted	Count
J	I	10
2	V	9
M	S	9
N	S	8
K	V	7
E	S	7
7	3	6
8	7	5

Sample misclassifications

Item	True	Predicted
P2_0_174	0	6
P5_0_426	0	6
P7_0_654	0	6
P8_0_751	0	B
P8_0_771	0	B
P9_0_865	0	6
P5_1_483	1	D
P5_1_485	1	D

gpt-5.6-terra

Most-confused pairs

True	Predicted	Count
M	S	10
1	D	10
J	I	10
F	W	9
2	V	9
P	G	9
N	S	9
9	W	8

Sample misclassifications

Item	True	Predicted
P3_0_239	0	6
P8_0_771	0	4
P10_1_928	1	D
P10_1_969	1	D
P2_1_138	1	D
P2_1_143	1	D
P5_1_483	1	D
P5_1_485	1	D

claude-opus-4.8**Most-confused pairs**

True	Predicted	Count
2	V	10
K	V	9
N	S	8
1	D	7
E	S	7
Q	X	6
T	S	6
M	S	5

Sample misclassifications

Item	True	Predicted
P2_0_174	0	7
P3_0_239	0	F
P8_0_751	0	1
P8_0_771	0	6
P9_0_845	0	B
P9_0_865	0	6
P10_1_928	1	L
P10_1_969	1	D

claude-sonnet-4.6**Most-confused pairs**

True	Predicted	Count
K	V	10
2	V	10
J	Y	8
E	S	8
D	1	8
S	A	6
3	V	6
P	G	6

Sample misclassifications

Item	True	Predicted
P1_0_21	0	9
P1_0_30	0	F
P2_0_174	0	B
P3_0_239	0	U
P5_0_426	0	F
P7_0_654	0	V
P8_0_751	0	B
P8_0_771	0	H